Python Notes — Conditional Statements

Topic: if / elif / else Logic

1. What Are Conditional Statements?

Conditional statements let your program make decisions. Based on whether a condition is True or False, Python will execute a different block of code. This is how programs react to data — like checking if someone is old enough, or if a day qualifies for a discount.

> 💡 **Tip:** Think of it like a bouncer at a door: 'If you're over 18, you can enter. Otherwise, go home.'

2. The if Statement

The if keyword checks a condition. If the condition is True, the indented block below it runs. If False, Python skips it entirely.

Syntax

```
if condition:
    # code to run if condition is True
```

Example

```
age = 25

if age >= 18:
    print("You are an adult")
```

Here, since age is 25 and 25 >= 18 is True, the print statement runs.

> ⚠️ **Warning:** Python uses indentation (4 spaces or 1 tab) to define the if block. If the indentation is missing or inconsistent, you will get an IndentationError.

3. The else Statement

else provides a fallback — code that runs when the if condition is False. There is no condition after else; it simply catches everything that didn't match.

```
age = 10

if age >= 18:
    print("adult")
else:
    print("not an adult")

# Output: not an adult
```

📝 **Note:** else is optional — you don't have to include it. But if you do, it must come after all if / elif blocks.

4. The elif Statement

elif (short for 'else if') lets you check multiple conditions in sequence. Python checks each one top-to-bottom and runs the first block whose condition is True, then skips the rest.

Syntax

```
if condition_1:
    # runs if condition_1 is True
elif condition_2:
    # runs if condition_2 is True (and condition_1 was False)
elif condition_3:
    # runs if condition_3 is True (and above were False)
else:
    # runs if ALL conditions above were False
```

Full Example from Code

```
age = 25

if age < 13:
    print("child")        # age < 13 → False, skip
elif age < 20:
    print("teenager")     # age < 20 → False, skip
elif age < 60:
    print("adult")        # age < 60 → True!  ✓ runs
else:
    print("senior")       # skipped

# Output: adult
```

Python tested each condition from top to bottom. The moment it found elif age < 60 to be True (since 25 < 60), it printed "adult" and exited the whole if/elif/else block without checking the rest.

💡 **Tip:** Once Python finds the first True condition, it runs that block and stops. Even if a later condition would also be True, it is never checked.

5. How Python Evaluates Conditions

Conditions use comparison operators that return True or False:

Operator	Meaning	Example
`==`	equal to	age == 25 → True
`!=`	not equal to	age != 30 → True
`<`	less than	age < 13 → False
`>`	greater than	age > 18 → True
`<=`	less than or equal	age <= 25 → True
`>=`	greater than or equal	age >= 18 → True

⚠️ **Warning:** Don't confuse = (assignment) with == (comparison). Writing if age = 25 causes a SyntaxError. Always use == inside conditions.

6. Combining Conditions

You can combine multiple conditions in one line using logical operators:

- and — both conditions must be True
- or — at least one condition must be True
- not — reverses the result

```
age = 25
day = "wednesday"

if age >= 18 and day == "wednesday":
    print("Adult on a Wednesday")

# Output: Adult on a Wednesday
```

7. Nested if Statements

You can place an if statement inside another if block. This is called nesting. Use it when the second check only makes sense if the first one passes.

```
age = 25
has_id = True

if age >= 18:
    if has_id:
        print("Entry allowed")
    else:
        print("Need ID")
else:
    print("Too young")

# Output: Entry allowed
```

📝 **Note:** Deeply nested conditions can become hard to read. If you have more than 2 levels of nesting, consider using 'and' to combine conditions instead.

8. Practical Example — Ticket Pricing

Here's the full ticket pricing logic from the session, broken down:

```
age = 22
day = "wednesday"

# Ternary: set base price based on age
price = 12 if age >= 18 else 8

# Conditional: apply Wednesday discount
if day == "wednesday":
    price -= 2

print("ticket price for you is: $", price)

# Output: ticket price for you is: $ 10
```

Step-by-step walkthrough:

- age = 22, so age >= 18 is True → price is set to 12
- day == "wednesday" is True → price -= 2 → price becomes 10
- Final output: ticket price for you is: $ 10

9. Quick Reference

Keyword	Meaning	When it runs
`if`	Starts the conditional block	When its condition is True
`elif`	Checks another condition	When all above conditions were False
`else`	Fallback block	When ALL conditions above were False
`and`	Both conditions must be True	Inside a condition expression
`or`	At least one must be True	Inside a condition expression
`not`	Reverses True/False	Inside a condition expression

10. Common Errors to Watch Out For

⚠️ **Error 1:** Using = instead of == in conditions → SyntaxError. Use == to compare.

⚠️ **Error 2:** Missing colon : at end of if/elif/else line → SyntaxError.

⚠️ **Error 3:** Incorrect indentation inside the block → IndentationError.

⚠️ **Error 4:** Writing elif or else without a preceding if → SyntaxError.